Lógica de programação e o primeiro programa

IAL-003 — Algoritmos e Programação de Computadores

Aula 01 - Apresentação da disciplina e critérios de avaliação. Introdução à Lógica de Programação.

Informática para Negócios, Fatec Mauá

Prof. Anderson Silva Vanin · 28 de agosto de 2026

Quem conduz a disciplina

Atuação multi-institucional

Docência e pesquisa na ETEC MCM (CPS), no Ibmec (Ciência de Dados & IA), na Fundação Santo André e na Fatec Mauá.

Pesquisa científica

Mestrado em Informática e Gestão do Conhecimento (UNINOVE), com foco em visão computacional e cidades inteligentes.

Inovação tecnológica

Coordenador de projetos na Assessoria INOVA (Centro Paula Souza) desde 2026 e dos laboratórios de IA e IoT desde 2022.

Academia

Publicações em periódicos internacionais, entre eles *Applied Soft Computing* (Elsevier).

Roteiro da aula

01 A disciplina e a avaliação

02 Lógica de programação e algoritmos

03 O ambiente Google Colab

04 print() e variáveis

05 Exercícios práticos

A disciplina e a avaliação

01

A disciplina ensina a analisar problemas e projetar soluções

Método para o desenvolvimento de algoritmos
Modularidade e abstração
Tipos de dados básicos e expressões aritméticas, lógicas e literais
Sequência, seleção simples e múltipla, iteração

Toda a prática acontece em Python, no navegador

Python 3

Linguagem de alto nível, com sintaxe próxima da linguagem natural. É a porta de entrada mais curta entre a ideia e o programa rodando.

Google Colab

Ambiente gratuito na nuvem. Nada para instalar: um navegador e uma conta Google bastam, e o arquivo fica salvo no seu Drive.

Google Drive

Uma pasta por aula, um notebook por encontro. O histórico do semestre inteiro fica organizado e recuperável na véspera da prova.

GitHub

O repositório da disciplina: notebooks e listas ficam publicados lá, e o seu portfólio de código começa a ser construído neste semestre.

A nota vem de três provas e de um projeto final

AVALIAÇÃO	CONTEÚDO COBRADO	DATA PREVISTA
P1	Lógica, tipos de dados, expressões, sequência e seleção	**16/10/2026**
P2	Repetição, listas, matrizes, strings e funções	**13/11/2026**
P3	Entrega e apresentação do projeto final estruturado	04/12/2026
Contínua	Listas de exercícios e projeto prático integrador	**Todo o semestre**

Lógica de programação é cumulativa

Traga as dúvidas da semana para o início da aula seguinte
Salve todo notebook no Drive, em uma pasta por aula
Refaça os exercícios sem olhar a solução — **ler código não ensina a escrever código**
Não existe "aula perdida": as reposições estão no cronograma oficial

28

Horas-aula a repor no semestre — doze das aulas de agosto e dezesseis das ausências já previstas. Todas voltam em catorze encontros extras de duas horas-aula, sempre às terças e quartas, das 7h40 às 9h20.

As aulas de agosto voltam em setembro e outubro

Aulas a repor

07/08 — sexta · 4 H/A

14/08 — sexta · 4 H/A

21/08 — sexta · 4 H/A

TOTAL 12 H/A

Datas de reposição

01/09 — terça

02/09 — quarta

08/09 — terça

09/09 — quarta

13/10 — terça

14/10 — quarta

6 ENCONTROS DE 2 H/A · TOTAL 12 H/A

As ausências de setembro e outubro voltam até novembro

Ausências previstas

18/09 — sexta · 4 H/A
25/09 — sexta · 4 H/A
02/10 — sexta · 4 H/A
09/10 — sexta · 4 H/A

TOTAL 16 H/A

Datas de reposição

20/10 — terça
21/10 — quarta
27/10 — terça
03/11 — terça
04/11 — quarta
10/11 — terça
11/11 — quarta
17/11 — terça

8 ENCONTROS DE 2 H/A ·

TOTAL 16 H/A

Toda reposição é às terças ou quartas, das 7h40 às 9h20

Catorze encontros extras de 2 H/A cobrem as 28 horas-aula — nenhum conteúdo sai do programa.
A presença nas reposições é registrada como em qualquer aula do semestre.
As reposições de 13/10 e 14/10 antecedem a P1; as de novembro cercam a P2.
Anote as catorze datas hoje: elas não se repetem no calendário da sexta-feira.

Lógica de programação e algoritmos

02

Um algoritmo é uma sequência finita e ordenada de passos

Executados nessa ordem, os passos resolvem um problema ou realizam uma tarefa. O conceito não é exclusivo da computação.

Uma receita de bolo é um algoritmo

O manual de montagem de um móvel é um algoritmo

Quatro critérios separam um algoritmo de uma intenção vaga

Finitude

Precisa terminar. Um conjunto de passos que nunca chega ao fim não resolve problema nenhum — é um travamento.

Precisão

Cada passo é claro e sem ambiguidade. "Bata a massa" não diz quanto tempo; o computador não preenche lacunas.

Entradas e saídas

O que o algoritmo recebe e o que ele produz precisam estar declarados antes de qualquer linha de código.

Eficácia

Toda instrução precisa ser executável. "Divida por zero" está escrito com precisão, mas não pode ser cumprido.

Trocar uma lâmpada tem sete passos

01 Desligar o interruptor

02 Pegar uma escada

03 Subir na escada

04 Retirar a lâmpada queimada

05 Colocar a lâmpada nova

06 Descer da escada

07 Ligar o interruptor

Cada passo é uma instrução única, na ordem em que precisa ser executada.

“Inverta os passos 1 e 4 e o algoritmo continua completo — mas agora ele dá choque.”

— a ordem dos passos faz parte do significado do algoritmo

A mesma ideia tem quatro representações

REPRESENTAÇÃO	O QUE É
Algoritmo	A sequência lógica de passos — independe de linguagem
Pseudocódigo	O algoritmo escrito em linguagem humana, mas estruturada
Fluxograma	A representação gráfica, com símbolos padronizados
Programa	O algoritmo em uma linguagem que o computador executa

Python aproxima o código da forma como pensamos o passo

Alto nível

A linguagem cuida da memória, dos tipos e do detalhe da máquina. Você descreve o passo; o interpretador resolve como executá-lo. Isso deixa a atenção onde ela importa nesta disciplina: na lógica.

A mesma linguagem aparece em ciência de dados, automação e desenvolvimento web — o que aprendemos aqui não fica no semestre.

Sintaxe enxuta

Uma linha de Python imprime na tela. Menos cerimônia significa menos erros que não têm relação com o problema — e um retorno visual mais rápido para quem está aprendendo a pensar em passos.

O ambiente Google Colab

03

O Colab executa Python em um servidor do Google

Ambiente gratuito e na nuvem — nada é instalado no seu computador

Os arquivos são *notebooks*, com extensão `.ipynb`, salvos no Drive

O código roda no servidor; o navegador só mostra o resultado

O mesmo notebook abre em qualquer máquina onde você entrar na sua conta

Um notebook se divide em células de texto e de código

Células de texto

Escritas em Markdown: títulos, listas, tabelas e explicações. Use-as para registrar o enunciado do exercício e o raciocínio antes da solução — é o seu caderno de anotações dentro do próprio arquivo.

O número à esquerda da célula — `[1]`, `[2]`, ... — indica a ordem em que ela foi executada.

Células de código

Onde os comandos Python são escritos e executados. Cada célula devolve o seu resultado logo abaixo dela, o que permite testar uma ideia por vez em vez de rodar o programa inteiro.

Texto e código se alternam no mesmo notebook

TEXTO

Aula 1 — Primeiro programa

Enunciados e anotações ficam aqui.

CÓDIGO

```
# meu primeiro programa
print("Olá, turma!")
```

[1]

```
Olá, turma!
```

Shift + Enter executa a célula

CÉLULA DE CÓDIGO

```
# Pressione Shift + Enter print("Colab funcionando!")
```

SAÍDA

```
Colab funcionando!
```

Ctrl +Enter executa e permanece na mesma célula — útil para testar variações da mesma linha.

Quatro atalhos resolvem o dia a dia no notebook

ATALHO	AÇÃO
`Shift + Enter`	Executa a célula e vai para a próxima
`Ctrl + Enter`	Executa a célula e permanece nela
`Ctrl + M B`	Insere uma célula abaixo
`Ctrl + M D D`	Exclui a célula



O Colab permite executar células fora de ordem, e o resultado depende da ordem real de execução — não da posição na tela. Quando algo não faz sentido, reinicie o ambiente e execute tudo de novo, de cima para baixo.

print() e variáveis

04

print() exibe informações na tela

CÓDIGO

```
print("Hello, World!")
```

SAÍDA

```
Hello, World!
```

O texto entre aspas é uma *string*. Os parênteses são a chamada da função; o que está dentro deles é o que será exibido.

A vírgula imprime vários valores em uma linha

CÓDIGO

```
print("Bem-vindo à disciplina!" )
print(2026)
print( "Turma:", "Informática", "- 2° ciclo")
```

SAÍDA

```
Bem-vindo à disciplina!
2026
Turma: Informática - 2° ciclo
```

Cada `print()` escreve uma linha; a vírgula separa valores dentro da mesma linha, com um espaço automático.

Uma variável é uma caixa com etiqueta na memória

A FORMA GERAL DA ATRIBUIÇÃO

```
variavel = valor
```

O sinal = não significa igualdade: ele guarda o valor da direita no nome da esquerda.

O Python identifica o tipo automaticamente

CÓDIGO

```
nome = "Ana" idade = 20
altura = 1.68

print("Nome:" , nome)
print("Idade:" , idade)
print("Altura:", altura)
```

SAÍDA

```
Nome: Ana
Idade: 20
Altura: 1.68
```

Seis regras governam os nomes de variáveis

Letras, números e `_` são permitidos, mas o nome não começa com número

Espaços não são permitidos: `nome_completo`, nunca `nome completo`

Maiúsculas contam: `idade`, `Idade` e `IDADE` são três variáveis

Evite acentos e caracteres especiais

Palavras reservadas estão fora: `print`, `if`, `for`, `while`

Por convenção, use `snake_case` — minúsculas separadas por `_`

O valor de uma variável pode mudar durante o programa

CÓDIGO

```
pontuacao = 0
print("Inicial:" , pontuacao)

pontuacao = 10
print("1ª rodada:" , pontuacao)

pontuacao = pontuacao + 5
print("2ª rodada:", pontuacao)
```

SAÍDA

```
Inicial: 0
1ª rodada: 10
2ª rodada: 15
```

Três mensagens de erro vão aparecer hoje

SyntaxError

A frase está mal formada: falta um parêntese ou uma aspa. O Python aponta a linha onde percebeu o problema — o erro costuma estar nela ou na anterior.

NameError

Você usou um nome que ainda não existe. Em geral é erro de digitação, diferença de maiúsculas, ou a célula que criava a variável não foi executada.

TypeError

A operação não vale para esses tipos. Somar texto com número é o caso clássico: `"idade" + 20` não é uma conta que o Python possa fazer.

Leia sempre a última linha da mensagem: é ali que está o tipo do erro e a explicação.

Exercícios práticos

05

Exercícios 1 e 2 — print() e três variáveis

Exercício 1

Escreva um programa que exiba o seu nome completo na tela usando `print()`.

Exercício 2

Crie as variáveis `nome`, `curso` e `periodo` com os seus dados e exiba as três em uma única chamada de `print()`, no formato do modelo abaixo.

SAÍDA ESPERADA — EXERCÍCIO 2

```
Nome: Ana Souza | Curso: Informática para Negócios | Período: Noite
```

Exercícios 3 e 4 — reatribuição e cartão de apresentação

Exercício 3

Crie a variável `idade` com a sua idade atual. Some 1 a ela, simulando um aniversário, e exiba o valor antes e depois da soma.

Resolva em um notebook novo, executando cada célula com `Shift + Enter` para conferir o resultado.

Exercício 4 — desafio

Usando apenas variáveis e `print()`, monte um cartão de apresentação com pelo menos quatro informações suas, cada uma em uma linha diferente.

Exercício 5 — placar de três rodadas

O que o programa faz

Crie as variáveis `jogador`, `rodada_1`, `rodada_2` e `rodada_3`. Calcule o `total` somando as três rodadas e uma variável `bonus`, que vale 5 pontos. Exiba o placar linha por linha e o total no fim.

As regras do desafio

Use apenas variáveis, atribuição e `print()` — nada além do conteúdo de hoje. O total precisa ser calculado a partir das variáveis, nunca digitado à mão: mudar uma rodada deve mudar o total sem editar mais nenhuma linha.

SAÍDA ESPERADA

```
Jogador: Ana | Rodadas: 10 8 12 | Bônus: 5
Total: 35
```

A entrega é o notebook salvo no seu Drive

Arquivo → Salvar uma cópia no Drive

Renomeie o arquivo como `Aula01_SeuNome.ipynb`

Envie o link conforme a orientação dada em sala

Guarde o arquivo na pasta da disciplina — ele será a sua referência na P1

Resumo da aula em quatro frases

Um algoritmo é uma sequência finita e ordenada de passos, e a ordem faz parte do significado

O Colab organiza o trabalho em células de texto e de código, executadas com `Shift + Enter`

`print()` exibe informações na tela; a vírgula reúne vários valores em uma linha

Variáveis guardam valores na memória, com tipagem dinâmica e regras próprias de nomenclatura

Referências desta aula

Básica

FORBELLONE, L. V.; EBERSPACHER, H. F. ***Lógica de programação: a construção de algoritmos e estruturas de dados.*** São Paulo: Prentice Hall, 2005. — capítulos iniciais: conceito de algoritmo e sequência.

ARAUJO, E. C. de. ***Algoritmos: fundamento e prática***. Visual Books, 2007. — características de um algoritmo e exemplos do dia a dia.

ASCENCIO, A. F. G.; CAMPOS, E. A. V. ***Fundamentos da programação de computadores***. São Paulo: Longman, 2007. — variáveis, tipos e comandos de saída.

Complementar e eletrônica

DOWNEY, A. ***Think Python***. GNU Free Documentation License, 2008. — cap. 1 e 2: o primeiro programa, variáveis e nomes.

DOWNEY, A.; ELKNER, J.; MEYERS, C. ***Como pensar como um cientista da computação***. GNU. — versão em português do mesmo material.

As obras sobre linguagens de programação e sobre C — Sebesta, Friedman, Silva & Melo e Feofiloff — serão referência nas aulas de estruturas e de paradigmas, mais adiante no semestre.

Próxima aula: tipos de dados e input()

Vamos estudar `int`, `float`, `str` e `bool`, e capturar dados digitados pelo usuário com a função `input()`.